Redes neuronales de visión del robot

Qué modelos corren en la Jetson Orin Nano durante las pruebas de campo, cómo está construida cada red, cuánto cuestan y qué conviene optimizar.

2 arquitecturas
YOLOv8s y YOLO11s

3 modelos
contexto + uno por cultivo

9,4 – 11,1 M
parámetros por modelo

49 GFLOPs
el más caro: ARANDANOS a 960 px

1. Qué corre en el robot

En cada cuadro de la cámara el robot ejecuta **dos detectores**: el modelo de **contexto**, que siempre está activo y encuentra plantas, macetas, personas y obstáculos, y el modelo del **cultivo** elegido en el dashboard: PLAGASX para coles o ARANDANOS para arándano. Los tres son detectores de una sola etapa de la familia YOLO de Ultralytics, entrenados en Kaggle con dos GPU T4. ARANDANOS y contexto comparten exactamente la misma red (YOLO11s); solo cambian las clases de salida y los pesos aprendidos.

	PLAGASX	ARANDANOS	Contexto
Para qué	enfermedades de la col (11 clases)	frutos maduro/inmaduro + 8 plagas (10 clases)	plantas, col, macetas, personas, obstáculos (9 clases)
Arquitectura	YOLOv8s	YOLO11s	YOLO11s
Parámetros	11.14 M	9.43 M	9.43 M
GFLOPs a 640 px	28.7	21.7	21.7
Tamaño en el robot	480 px	960 px	640 px
GFLOPs en el robot	16.1	49.1	21.7
Tiempo medido en la Jetson*	41 ms	135 ms	63 ms

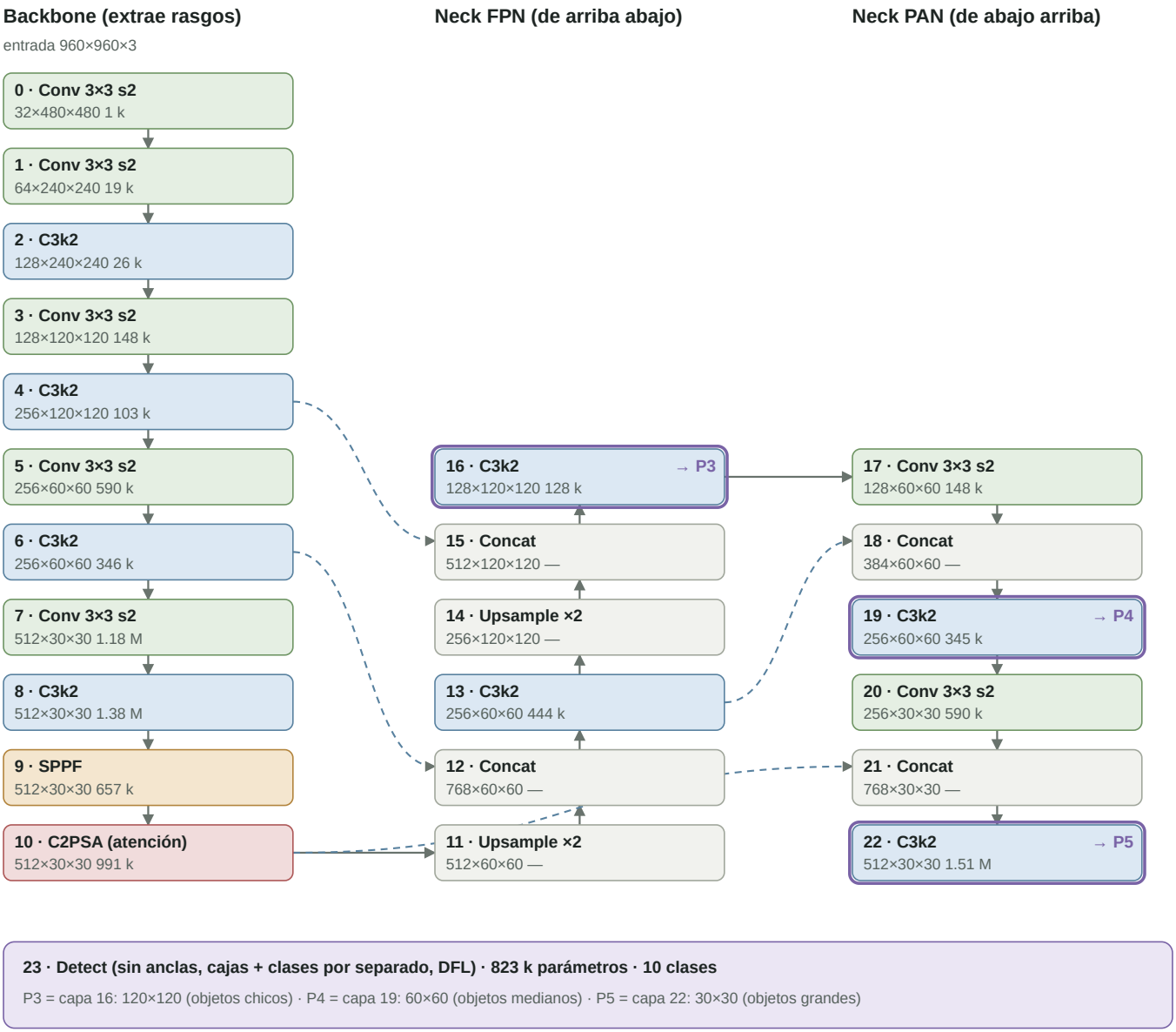
	PLAGASX	ARANDANOS	Contexto
Entrenamiento	Kaggle, 80 épocas, 800 px (28-oct)	Kaggle, 157 épocas, 960 px (11-nov)	Kaggle, 81 épocas, 800 px (9-nov)
Validación (mAP50 / mAP50-95)	0,83 / 0,66	0,79 / 0,46	0,68 / 0,44
Cuándo corre	con el perfil «Coles»	con el perfil «Arándano»	siempre

*Mediana de 25 cuadros reales de 640×480, en PyTorch FP32 y modo de 15 W, con el backend del robot usando la GPU al mismo tiempo (condición real de campo). Incluye preproceso y postproceso.

2. Cómo es cada red

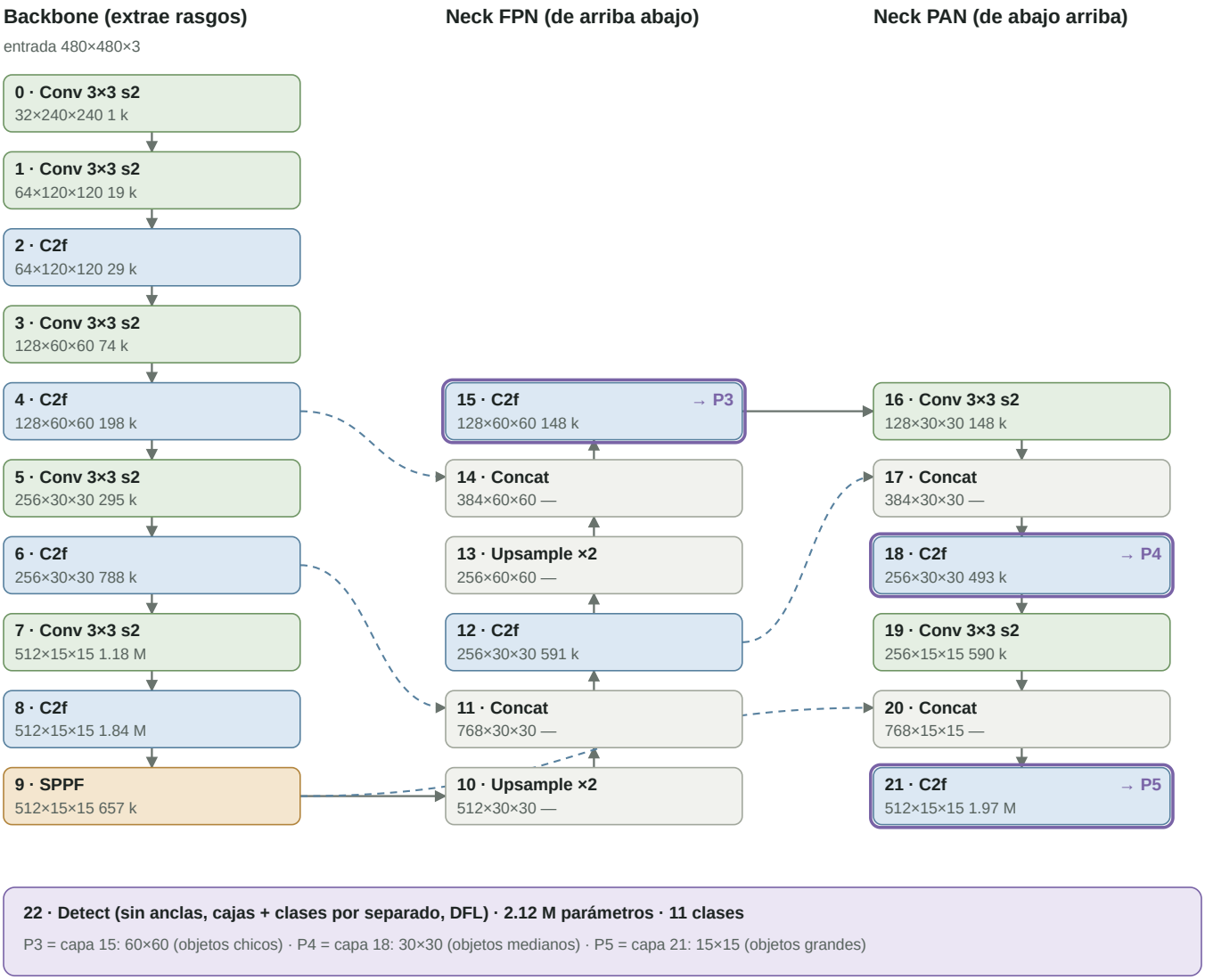
Todas siguen el mismo esquema de tres partes. El **backbone** reduce la imagen a la mitad cinco veces (strides 2 a 32) y extrae rasgos cada vez más abstractos. El **neck** mezcla esos rasgos en dos pasadas: una de arriba abajo (FPN), que lleva el contexto a las capas finas, y otra de abajo arriba (PAN), que devuelve el detalle a las capas gruesas. La **cabeza Detect** predice cajas y clases en tres escalas: P3 para objetos chicos (insectos, frutos), P4 para medianos y P5 para grandes (plantas, personas). Los diagramas se generaron a partir de las capas reales de cada archivo .pt, con el tamaño de salida que tiene cada capa a la resolución usada en el robot.

YOLO11s: ARANDANOS (960 px) y contexto (640 px)



Capas 0–10: backbone. 11–22: neck. 23: cabeza. Cada caja muestra la salida (canales × alto × ancho) a 960 px y los parámetros de la capa. Las líneas punteadas son conexiones de salto (Concat). El contexto tiene la misma estructura con salidas de 80×80, 40×40 y 20×20.

YOLOv8s: PLAGASX (480 px)

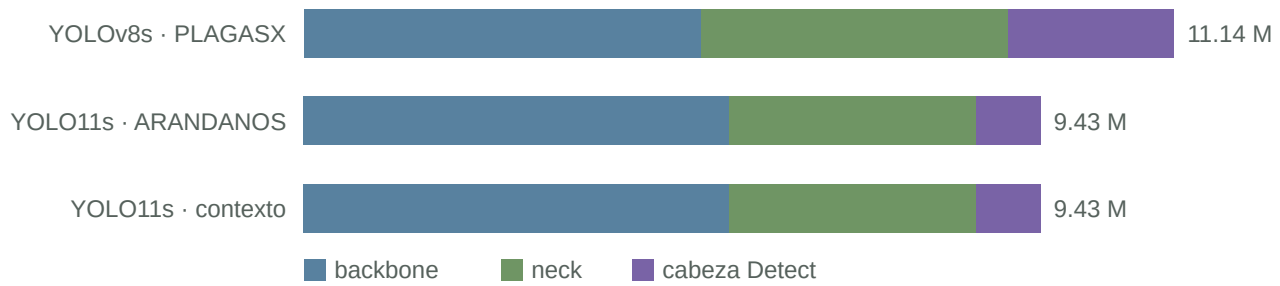


Capas 0–9: backbone. 10–21: neck. 22: cabeza. Salidas a 480 px: P3 de 60×60, P4 de 30×30 y P5 de 15×15.

Bloques que forman las redes

Bloque	Qué hace	Red
Conv	convolución 3×3 + BatchNorm + SiLU; con stride 2 reduce el tamaño a la mitad	las dos
C2f	divide los canales, pasa una parte por varios cuellos de botella y concatena todas las salidas intermedias: más flujo de gradiente con pocos parámetros	YOLOv8s
C3k2	sucesor de C2f con bloques C3k (núcleos 3×3 en serie); misma función con menos parámetros	YOLO11s

Bloque	Qué hace	Red
SPPF	pooling espacial piramidal rápido: tres max-pool 5×5 encadenados que amplían el campo receptivo sin reducir el tamaño	las dos
C2PSA	bloque de atención espacial parcial: deja que cada zona de la imagen pondere a las demás; ayuda con objetos chicos en fondos con mucho follaje	solo YOLO11s
Detect	cabeza sin anclas con ramas separadas para cajas y clases; la caja se predice como distribución (DFL)	las dos



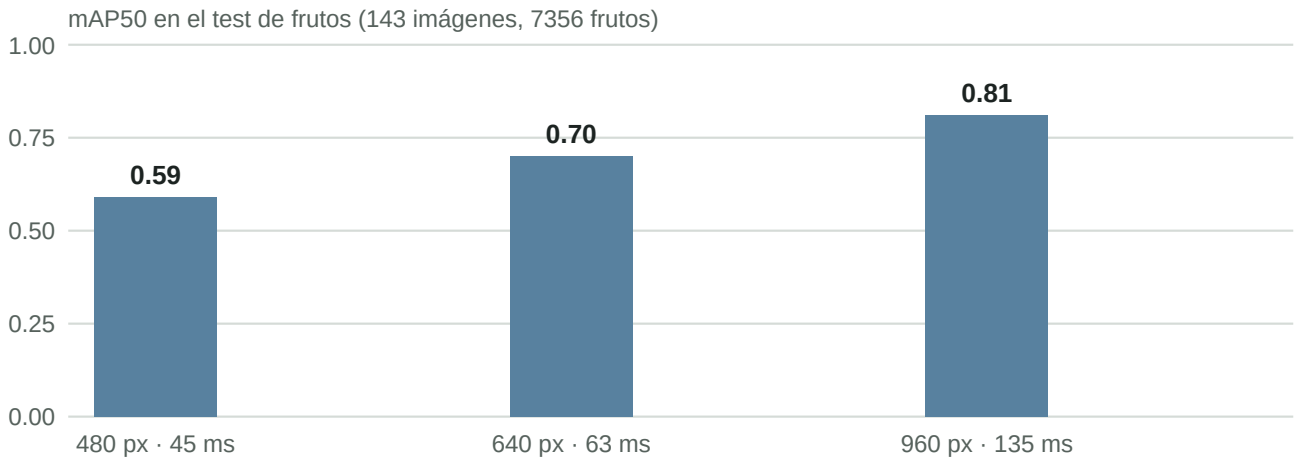
Dónde están los parámetros. YOLO11s pone más peso en la atención (C2PSA) y aligera la cabeza: 0,82 M frente a 2,12 M de YOLOv8s.

3. Rendimiento medido en la Jetson

Modelo	Tamaño (px)	FP32 (ms)	FP16 (ms)	En el robot
contexto (YOLO11s)	640	63	52	en uso
contexto (YOLO11s)	480	46	48	
PLAGASX (YOLOv8s)	480	41	41	en uso
PLAGASX (YOLOv8s)	640	52	55	
ARANDANOS (YOLO11s)	960	135	137	en uso
ARANDANOS (YOLO11s)	640	63	63	
ARANDANOS (YOLO11s)	480	45	36	

Dos conclusiones. Primera: en PyTorch, **FP16 casi no acelera** en esta Jetson. La versión instalada de PyTorch no trae núcleos compilados para la arquitectura del Orin

(capacidad 8.7), así que el cálculo en media precisión no aprovecha los Tensor Cores.
Segunda: **el tamaño de entrada manda**. El costo crece con el cuadrado del lado de la imagen: ARANDANOS a 960 px hace 2,3 veces más operaciones que a 640 px.



Lo que cuesta achicar la imagen en ARANDANOS: a 640 px tarda la mitad, pero pierde 0,11 de mAP50; a 480 px pierde 0,22. Los frutos ocupan pocos píxeles y necesitan resolución.

Tamaño	mAP50	mAP50-95	Precisión	Recall	Tiempo
960 px	0.81	0.49	0.84	0.75	135 ms
640 px	0.70	0.41	0.80	0.64	63 ms
480 px	0.59	0.33	0.79	0.53	45 ms

4. Qué optimizar, en orden

	Acción	Por qué
1	Exportar los tres modelos a **TensorRT FP16**	Es la vía oficial de NVIDIA para Jetson y la que aprovecha los Tensor Cores que PyTorch hoy no usa. TensorRT no está instalado en esta JetPack (R39); hay que instalarlo y medir. Ultralytics lo hace con `model.export(format='engine', half=True)`.
2	Mantener 960 px en ARANDANOS y bajar el costo por otra vía	Achicar la imagen cuesta precisión (tabla anterior). Mejor: TensorRT, inferir solo cuando el robot está quieto frente a la planta, o usar el zoom de la cámara (recorte real de 1280×720) para que el fruto ocupe más píxeles.

	Acción	Por qué
3	Unir contexto y cultivo en un solo modelo	Las dos redes de un mismo perfil son YOLO11s. Un solo YOLO11s con las clases de ambos haría una inferencia por cuadro en vez de dos (ahorro cercano a la mitad del cómputo de visión).
4	Pasar PLAGASX de YOLOv8s a YOLO11s	Mismo tamaño de familia con 15 % menos parámetros y 24 % menos GFLOPs (21,7 frente a 28,7 a 640 px), más el bloque de atención. Hay que reentrenar y comparar el mAP.
5	Cuantización INT8 con calibración	Tras TensorRT FP16. Reduce memoria y tiempo; necesita imágenes de calibración del propio robot y verificar que no caiga el mAP.
6	Reentrenar con datos de campo	La mayor ganancia de calidad: las fotos de Vitro Plantas y de la huerta de coles como negativos y positivos etiquetados (ver informes de las pruebas).

> **Lo que no está medido.** Las ganancias de TensorRT, INT8 y de unir modelos son estimaciones de diseño; no se probaron porque TensorRT no está instalado. Para la tesis conviene presentarlas como trabajo futuro, o medirlas antes de la defensa.

5. Para la defensa

- **Por qué YOLO:** detector de una sola etapa, en tiempo real en hardware embebido, con exportación directa a TensorRT; el robot necesita cajas (dónde está la plaga o el fruto) y no solo una clase por imagen.
- **Por qué la escala «s»:** 9–11 M de parámetros y 20–30 GFLOPs a 640 px caben en la Orin Nano junto con SLAM y navegación, con decenas de milisegundos por cuadro.
- **Por qué YOLO11s en los modelos nuevos:** menos parámetros y operaciones que YOLOv8s, y el bloque C2PSA de atención, útil con objetos chicos sobre follaje.
- **Por qué 960 px en arándano:** medido: a 640 px el mAP50 cae de 0,81 a 0,70.
- **Limitación honesta:** los modelos de cultivo se entrenaron sobre todo con fotos públicas; en campo aparecieron falsos positivos (follaje lejano, hojas nuevas). Se corrigieron en parte con filtros y se atacan reentrenando con los datos recolectados.

Fuentes: los archivos `agrobot_contexto.pt`, `agrobot_plagasx_det.pt` y `agrobot_arandanos_det.pt` (arquitectura, parámetros y métricas de entrenamiento leídos de cada checkpoint); mediciones en la Jetson del 11-nov-2026; Ultralytics YOLOv8 (2023) y YOLO11 (2024), documentación oficial.